

Geez Security

Web Application Penetration Test

OWASP Juice Shop

Report of Findings

Report Information	Value
Application	OWASP Juice Shop
Target IP	10.248.139.102
Assessment Type	Authorized CTF / Web Application Security Assessment
Prepared By	Yabsira Dawit
Selected Findings	6
Primary Tools	Burp Suite, Browser DevTools
Contact information	Email: yabudave2@gmail.com

Contents

1. Document Control

2. Executive Summary

3. Technical Summary

4. Findings Overview

5. Technical Detail (6 Findings)

6. Summary of Findings

7. Conclusion

8. Appendices

Document Control

Confidentiality

In no event shall Geez Security (GTST) be liable to anyone for special, incidental, collateral, or consequential damages arising out of the use of this information. This document contains information which is confidential and proprietary to the assessed organization. Extreme care should be exercised before distributing copies of this document, or the extracted contents of this document. This document should be marked "CONFIDENTIAL" and disseminated on a need-to-know basis only.

Disclaimers

The information presented in this document is provided as-is and without warranty. Vulnerability assessments are a "point in time" analysis; it is possible that the environment has changed since the tests reflected in this report were run, or that new vulnerabilities have since been discovered. This report should be considered a guide, not a complete representation of risk to the assessed application.

Proprietary Information

The content of this document is proprietary and should not be disclosed outside the recipient organization's network. Geez Security grants permission to copy this report for the purpose of disseminating information internally or to a relevant regulatory body.

Contact Information

Name	Title	Contact Information
Geez Security		
Yabsira Dawit	GTST Student	Email: yabudave2@gmail.com
		Phone no: 0932663504
		Telegram: @yeab_nagen

Executive Summary

Geez Security performed a web application penetration test against the OWASP Juice Shop application, an intentionally vulnerable e-commerce platform used for security training, over the assessment period of August 15–19, 2026. Testing was conducted using a black-box to gray-box methodology, simulating an external attacker with no privileged access at the start of testing.

By leveraging a series of chained attacks — beginning with SQL injection against the login form — Geez Security demonstrated a full authentication bypass leading to administrative account access. From this position, further weaknesses were identified that would allow an attacker to access other users' data, delete content without authorization, and execute persistent script injection against application users.

It is strongly recommended that the identified critical and medium-severity issues be remediated as a priority, as several are exploitable with minimal technical skill and no authenticated starting access.

Assessment Summary

The current risk posture of the assessed application is rated CRITICAL. Two findings were rated Critical severity, both stemming from authentication weaknesses that grant full administrative access to an unauthenticated attacker. The remaining four findings were rated Medium severity. The table below summarizes findings by severity (informational items are excluded):

Phase	Description	Critical	Medium	Low	Total
1	Web Application Penetration Testing	2	4	0	6
	Total	**2**	**4**	**0**	**6**

Strategic Recommendation

We recommend addressing the CRITICAL findings immediately, followed by the MEDIUM findings as part of the next regular release cycle.

Technical Summary

Finding Severity Ratings

The table below defines the risk naming and CVSS v3.0 score ranges used throughout this report. Quantifying overall business risk is outside the scope of this assessment; a finding rated High from a technical standpoint may be considered acceptable by the business as a result of compensating controls not visible to the assessment team.

Severity	CVSS v3.0 Range	Definition
Critical	9.0 – 10.0	Exploitation is straightforward and usually results in system-level compromise. Form a plan of action and patch immediately.
High	7.0 – 8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data. Patch as soon as possible.
Medium	4.0 – 6.9	Vulnerabilities exist but require extra steps to exploit. Patch after high-priority issues are resolved.
Low	0.1 – 3.9	Non-exploitable but reduces attack surface. Patch during the next maintenance window.

Scope

Assessment	Details
Web Application Penetration Test	OWASP Juice Shop — http://10.48.139.102

Scope Exclusions

Per assessment scope, Geez Security did not perform any Denial of Service (DoS) attacks during testing.

Post-Assessment Clean-up

All test accounts created for the purpose of this assessment should be disabled or removed, together with any associated content, following remediation validation.

Findings Overview

All issues identified during the assessment are listed below with a brief description and CVSS-based severity rating.

Ref	Description	CVSS Score	Severity
JS-01	SQL Injection — Authentication Bypass	9.1	**CRITICAL**
JS-02	Broken Authentication — Brute-Forceable Password	9.1	**CRITICAL**
JS-03	Insecure Direct Object Reference (IDOR)	6.5	**MEDIUM**
JS-04	Broken Access Control — Unauthorized Deletion	6.5	**MEDIUM**

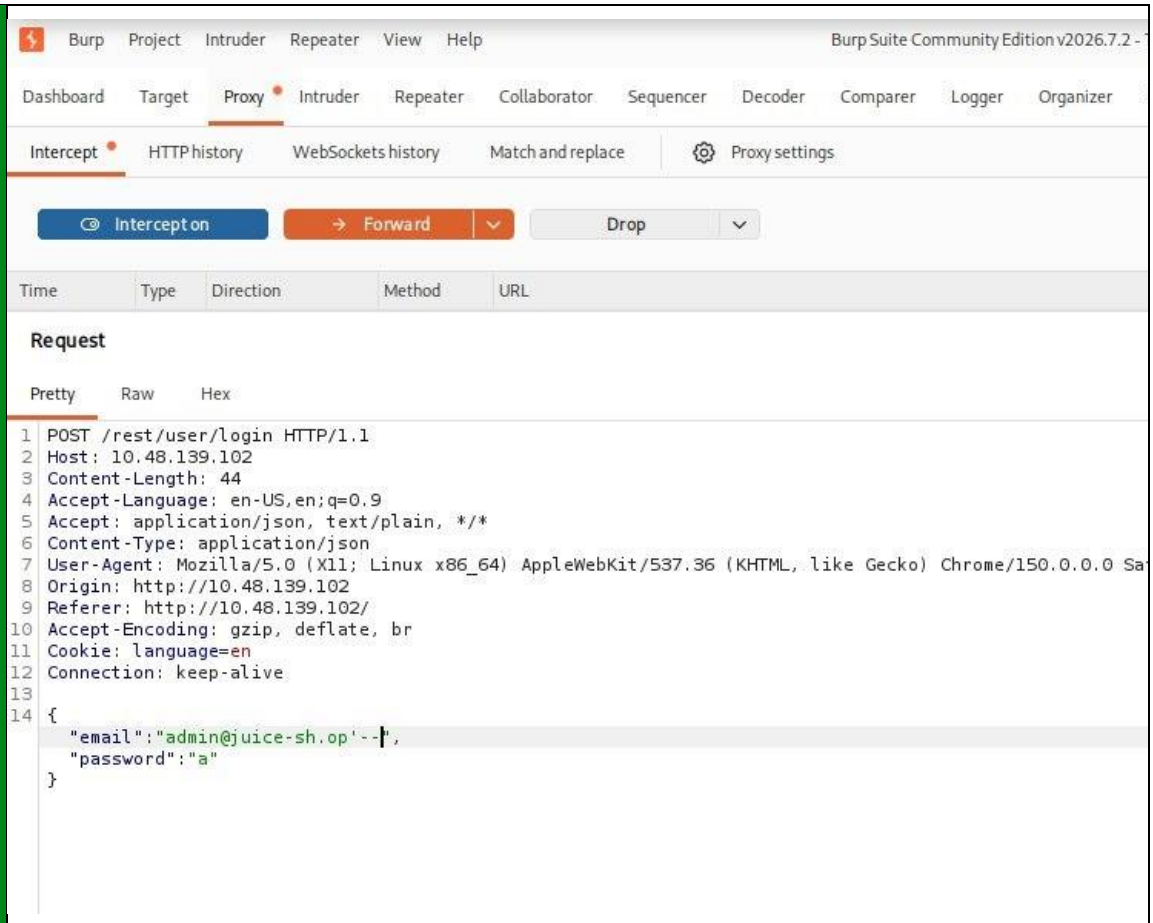
JS-05	Persistent Cross-Site Scripting (XSS)	6.1	**MEDIUM**
JS-06	Sensitive Information Disclosure	5.3	**MEDIUM**

Technical Detail

Each finding below documents the vulnerability, its CVSS v3.0 base score and vector (calculated per the FIRST.org CVSS v3.0 methodology), reproduction steps, business impact, and remediation guidance.

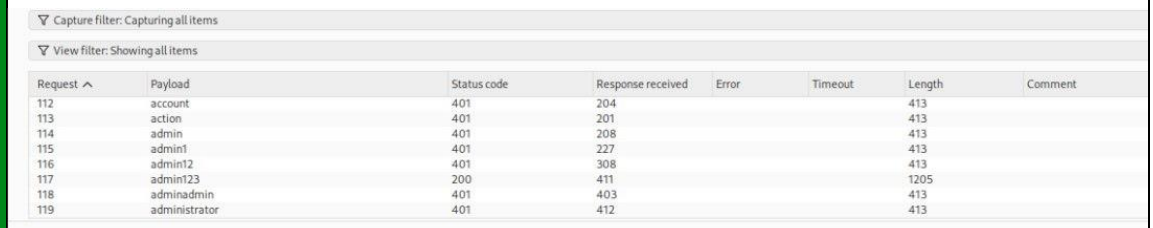
Finding 1: SQL Injection — Authentication Bypass

Ref ID	JS-01
Vulnerability Name	**SQL Injection — Authentication Bypass**
Risk / Severity	**CRITICAL**
CVSS v3.0 Score	**9.1 (CRITICAL)**
CVSS Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Vulnerability Explanation	**Affects:** Login endpoint (/rest/user/login) **Parameter(s):** email **Attack Vector:** Network — unauthenticated, no user interaction required **References:** CWE-89: SQL Injection; OWASP Top 10 2021 A03 (Injection)
Vulnerability Description	The login form's email field is concatenated directly into a backend SQL query without parameterization or input sanitization. Appending a single quote followed by a SQL comment sequence (' --) truncates the query before the password comparison, causing the database to authenticate the request based on the email match alone. The same technique succeeded against multiple accounts, including the administrator account, confirming the flaw is systemic to the login handler rather than specific to one user record.
Business Impact	This is the most severe finding in the assessment. It allows any unauthenticated attacker to fully bypass authentication and gain administrative access to the application without needing to know or guess any password. Once authenticated as an administrator, an attacker inherits every privilege available in the admin panel — including the access described in JS-04 — making this finding the root cause of much of the assessment's downstream impact.
Steps to Reproduce	**1.** Navigate to the application login page. **2.** In the Email field, enter: admin@juice-sh.op' -- **3.** Enter any value in the Password field. **4.** Submit the form. **5.** Observe successful authentication as the administrator account without a valid password.

Proof	
Recommendation	**Who: GTST student (Yabsira Dawit)** • Replace string-concatenated SQL queries with parameterized queries / prepared statements across all authentication logic. • Apply strict input validation on the email field, rejecting SQL metacharacters where they are not semantically valid. • Implement a Web Application Firewall (WAF) rule set to detect and block common SQL injection payloads as a compensating control. • Conduct a full code review of all database-facing input handlers, not just the login endpoint, to rule out the same pattern elsewhere.

Finding 2: Broken Authentication — Brute-Forceable Administrator Password

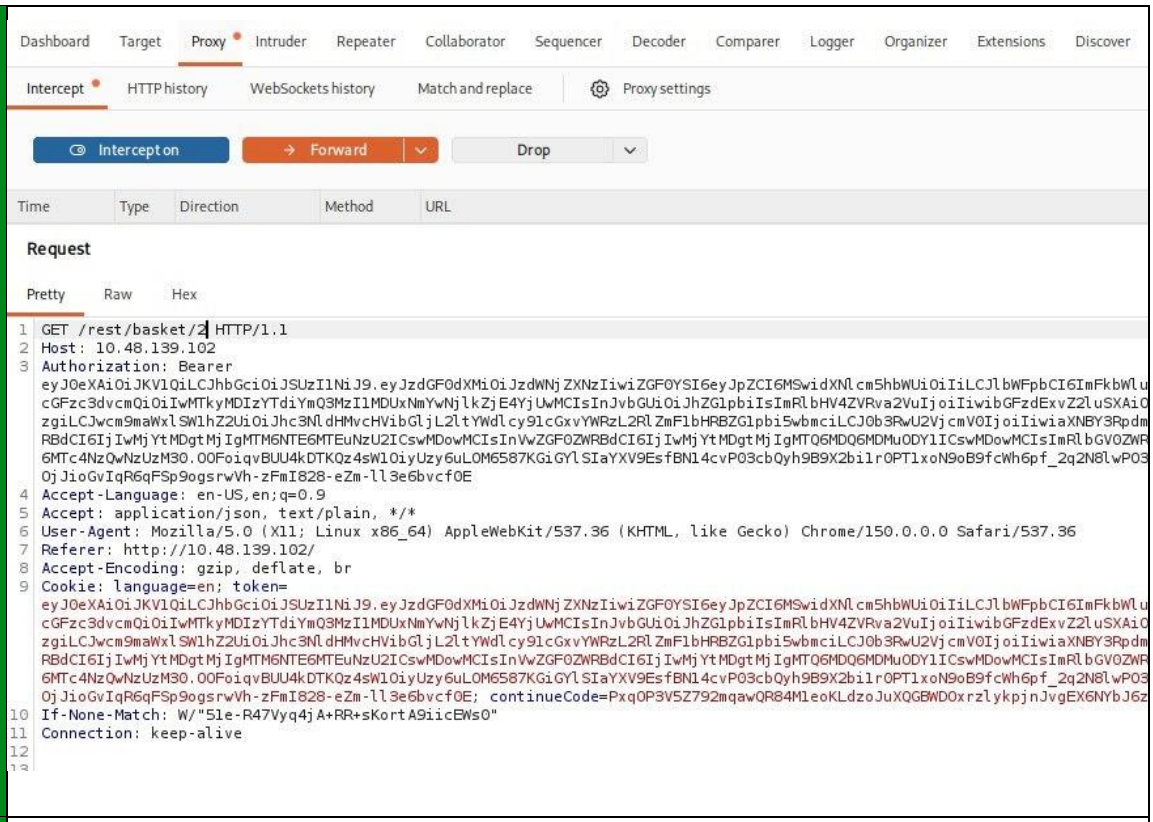
Ref ID	JS-02
Vulnerability Name	**Broken Authentication — Brute-Forceable Administrator Password**
Risk / Severity	**CRITICAL**
CVSS v3.0 Score	**9.1 (CRITICAL)**

CVSS Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Vulnerability Explanation	**Affects:** Login endpoint (/rest/user/login) **Parameter(s):** email, password **Attack Vector:** Network — unauthenticated, automatable **References:** CWE-307: Improper Restriction of Excessive Authentication Attempts; OWASP Top 10 2021 A07 (Identification and Authentication Failures)
Vulnerability Description	The login endpoint does not implement any account lockout, rate limiting, or CAPTCHA after repeated failed login attempts. Using Burp Suite's Intruder module with a common-password wordlist, the administrator account's password was recovered after an automated attack of approximately 1,050 attempts, with no throttling or blocking observed at any point during the attack.
Business Impact	Combined with the absence of multi-factor authentication, this finding means the administrator account is protected by password strength alone. An attacker with no prior access can obtain full administrative credentials using only a laptop and a publicly available wordlist, with no specialized exploit development required. This finding is independent of JS-01 — even if the SQL injection in JS-01 were fixed, this issue alone would still grant full account takeover.
Steps to Reproduce	**1.** Intercept a login request using Burp Suite and send it to Intruder. **2.** Set the payload position on the password parameter. **3.** Load a common-password wordlist (e.g. SecLists best1050.txt). **4.** Launch the attack against the admin account email. **5.** Filter results by HTTP response code; the request returning 200 OK identifies the valid password.
Proof	
Recommendation	**Who: GTST student (Yabsira Dawit)** • Implement account lockout or exponential back-off after a small number of failed login attempts (e.g. 5). • Enforce a strong password policy (minimum 14 characters, no dictionary words) per CIS benchmarks. • Add multi-factor authentication (MFA) for administrative accounts at minimum. • Deploy CAPTCHA or similar bot-mitigation on the login form after repeated failures from the same source.

Finding 3: Insecure Direct Object Reference (IDOR) — Basket Access

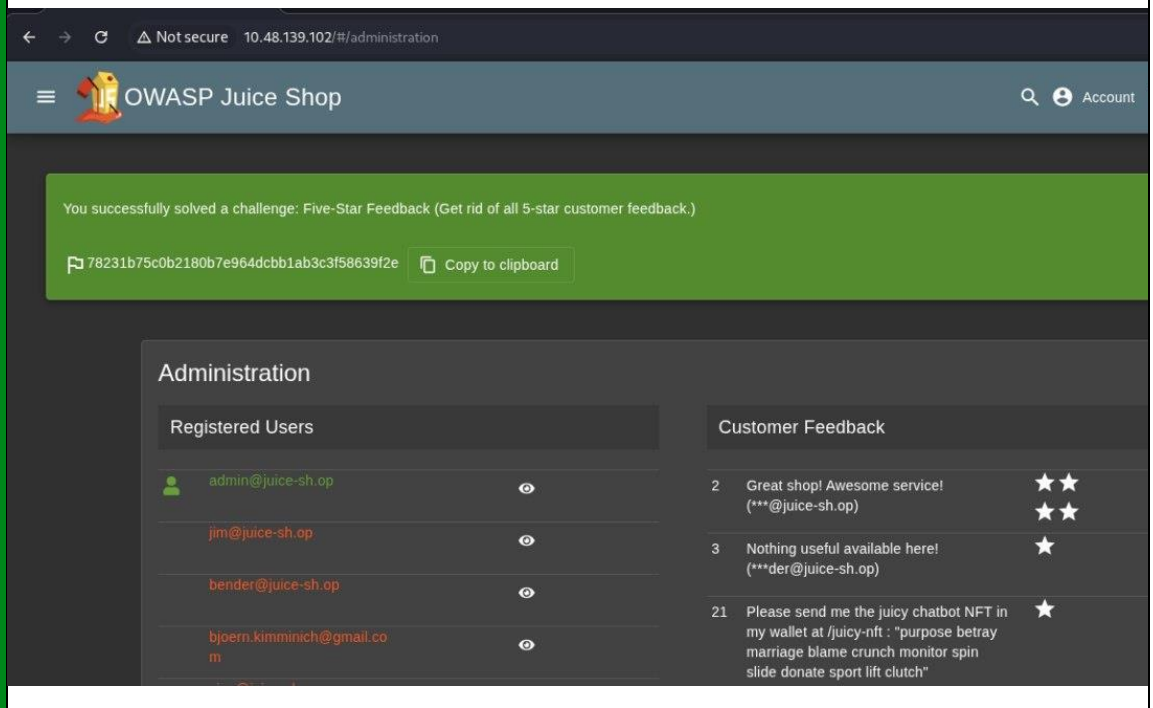
Ref ID	JS-03
Vulnerability Name	**Insecure Direct Object Reference (IDOR) — Basket Access**

Risk / Severity	**MEDIUM**
CVSS v3.0 Score	**6.5 (MEDIUM)**
CVSS Vector	CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Vulnerability Explanation	**Affects:** /rest/basket/{id} **Parameter(s):** id (URL path parameter) **Attack Vector:** Network — requires a low-privilege authenticated session **References:** CWE-639: Authorization Bypass Through User-Controlled Key; OWASP Top 10 2021 A01 (Broken Access Control)
Vulnerability Description	The shopping basket endpoint accepts a numeric basket ID directly in the URL and returns the associated basket contents without verifying that the requesting user is the basket's owner. By simply incrementing the ID in an intercepted request from 1 to 2, the basket contents belonging to a different user account were returned in full.
Business Impact	Any authenticated user — regardless of privilege level — can enumerate and view the shopping basket contents of every other user on the platform by iterating over basket IDs. While basket contents are not highly sensitive in isolation, this pattern typically indicates the same authorization gap exists on other ID-based endpoints (orders, addresses, payment methods), which would materially increase severity if confirmed.
Steps to Reproduce	**1.** Log in as any user (admin was used for verification). **2.** Enable Burp Suite Intercept and click "Your Basket". **3.** Observe the outgoing request: GET /rest/basket/1 **4.** Modify the request to GET /rest/basket/2 and forward it. **5.** Confirm the response contains basket contents belonging to a different account.

Proof	
Recommendation	• **Who: GTST student (Yabsira Dawit)** • Enforce server-side authorization checks that verify the authenticated session owns the requested basket ID before returning data. • Replace sequential integer IDs with non-guessable identifiers (e.g. UUIDs) as a defense-in-depth measure. • Apply the same ownership-verification pattern to all other user-scoped endpoints (orders, addresses, payment). • Add automated authorization tests to the CI pipeline covering cross-user object access.

Finding 4: Broken Access Control — Unauthorized Review Deletion

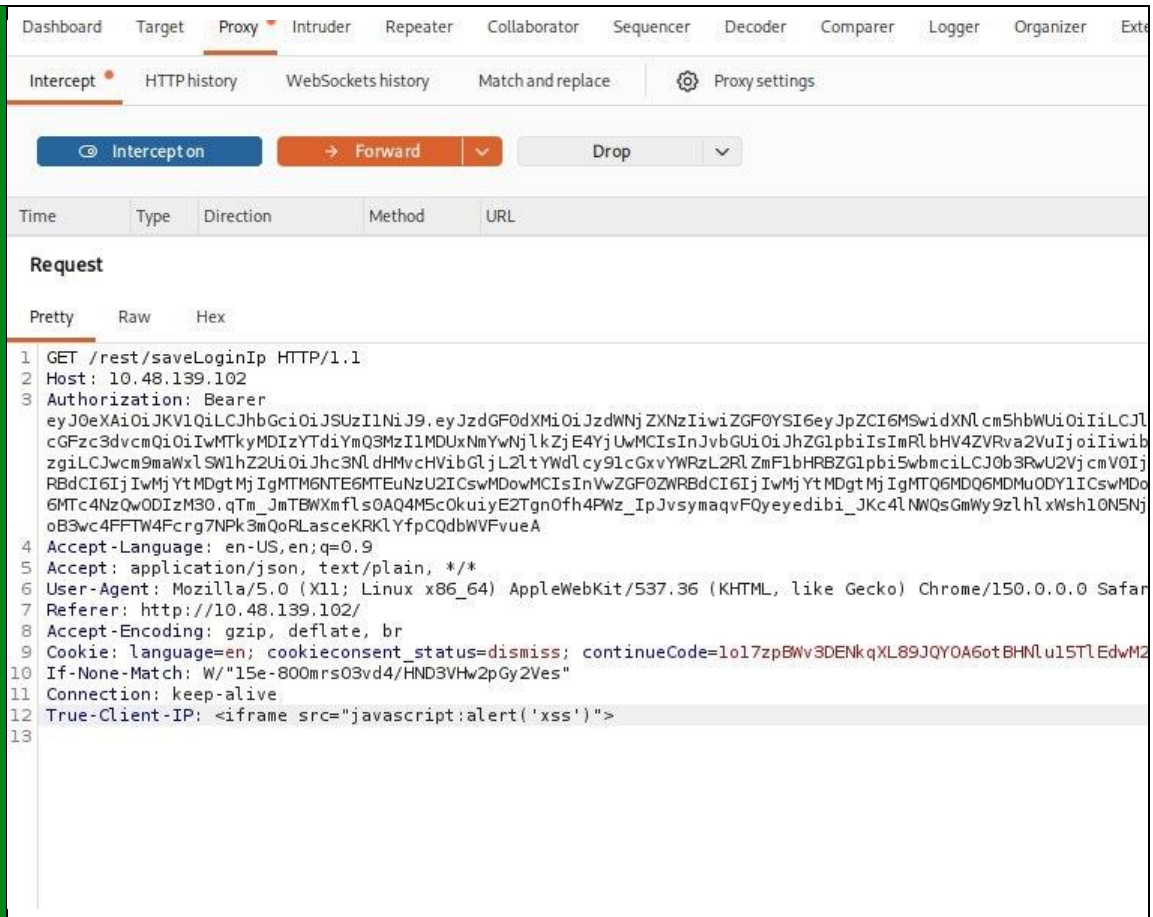
Ref ID	JS-04
Vulnerability Name	**Broken Access Control — Unauthorized Review Deletion**
Risk / Severity	**MEDIUM**
CVSS v3.0 Score	**6.5 (MEDIUM)**
CVSS Vector	CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
Vulnerability Explanation	**Affects:** Admin panel — product review deletion endpoint **Parameter(s):** review ID

	Attack Vector: Network — requires an authenticated session **References:** CWE-863: Incorrect Authorization; OWASP Top 10 2021 A01 (Broken Access Control)
Vulnerability Description	The administration panel's review-deletion function does not adequately restrict which reviews an authenticated user is permitted to delete. During testing, a five-star product review — including reviews not authored by the testing account — was deleted directly through the admin interface without any secondary confirmation, audit prompt, or ownership check beyond the initial admin-panel access.
Business Impact	While this finding requires admin-level access (itself achievable via JS-01 or JS-02), it demonstrates that once inside the admin panel there is no additional layer of authorization or audit control on destructive actions. An attacker or malicious insider could permanently remove legitimate customer feedback with no logging trail identified during testing, undermining the integrity of product review data.
Steps to Reproduce	**1.** Authenticate to the application as an administrator. **2.** Navigate to the Admin Panel. **3.** Locate the target five-star review in the review management list. **4.** Click the delete (trash) icon next to the review. **5.** Confirm the review is permanently removed with no additional authorization step.
Proof	
Recommendation	**Who: GTST student (Yabsira Dawit)** • Add a confirmation step and audit logging for all destructive actions performed through the admin panel. • Apply role-based access control (RBAC) so that content moderation actions are scoped and logged per administrator identity. • Retain deleted content in a soft-delete state for a defined retention period to allow recovery.

	• Review all other destructive admin-panel actions for the same missing-confirmation pattern.

Finding 5: Persistent Cross-Site Scripting (XSS) via HTTP Header

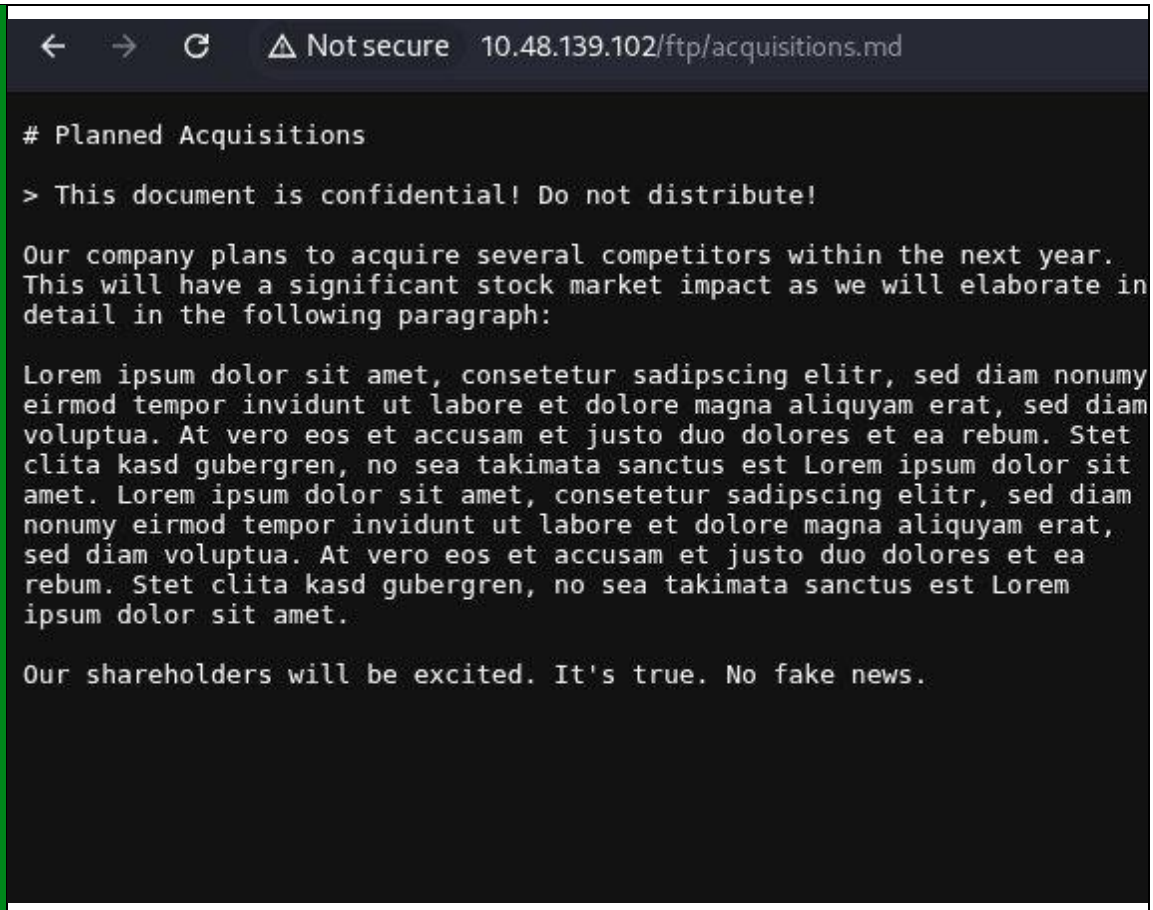
Ref ID	JS-05
Vulnerability Name	**Persistent Cross-Site Scripting (XSS) via HTTP Header**
Risk / Severity	**MEDIUM**
CVSS v3.0 Score	**6.1 (MEDIUM)**
CVSS Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Vulnerability Explanation	**Affects:** True-Client-IP request header, reflected on the "Last Login IP" profile field **Parameter(s):** True-Client-IP (HTTP header) **Attack Vector:** Network — requires a victim to view the affected page after the payload is stored **References:** CWE-79: Improper Neutralization of Input During Web Page Generation (XSS); OWASP Top 10 2021 A03 (Injection)
Vulnerability Description	The application logs the value of the True-Client-IP request header and later renders it, unescaped, on the user's "Last Login IP" profile field. Submitting a login request with an HTML/JavaScript payload in this header causes the payload to be stored server-side and executed in the browser of anyone who subsequently views that field — making this a persistent (stored) XSS rather than a one-time reflected injection.
Business Impact	Because the payload persists and requires no repeat injection, it can affect any user (including administrators) who later views the "Last Login IP" field — for example while auditing account activity. A realistic payload could exfiltrate session cookies or perform actions on behalf of the viewing user, extending the attacker's reach beyond the account that was originally logged into.
Steps to Reproduce	**1.** Authenticate to the application and enable Burp Suite Intercept. **2.** Log out to trigger a fresh login request. **3.** Add a custom header: True-Client-IP: <iframe src="javascript:alert('xss')"> **4.** Forward the modified request to complete the login. **5.** Log back in and navigate to the account's "Last Login IP" field to confirm payload execution.

Proof	
Recommendation	**Who: GTST student (Yabsira Dawit)** • Apply context-appropriate output encoding to all header-derived values before rendering them in HTML. • Treat all HTTP headers as untrusted input, including client-supplied headers such as True-Client-IP and X-Forwarded-For. • Implement a Content Security Policy (CSP) restricting inline script execution as a defense-in-depth control. • Add automated XSS regression tests covering header-derived rendering paths.

Finding 6: Sensitive Information Disclosure — Exposed Confidential Document

Ref ID	JS-06
Vulnerability Name	**Sensitive Information Disclosure — Exposed Confidential Document**
Risk / Severity	**MEDIUM**
CVSS v3.0 Score	**5.3 (MEDIUM)**

CVSS Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Vulnerability Explanation	**Affects:** /ftp/ directory **Parameter(s):** N/A — direct path navigation **Attack Vector:** Network — unauthenticated, no user interaction required **References:** CWE-200: Exposure of Sensitive Information to an Unauthorized Actor; OWASP Top 10 2021 A01 (Broken Access Control)
Vulnerability Description	An /ftp/ directory is reachable over HTTP without authentication and is not referenced anywhere in the application's normal navigation flow. It was discovered by following an incidental link on the About Us page and browsing the parent directory. The directory contains internal files, including a document named acquisitions.md, which appears to contain confidential business information not intended for public or customer access.
Business Impact	This finding represents a lower-complexity but still significant information disclosure issue: no authentication or specialized tooling is required, only knowledge that the path exists. Depending on the actual contents of files hosted in this directory, exposure could range from low-sensitivity internal documentation to material non-public business information.
Steps to Reproduce	**1.** Browse to the application's About Us page. **2.** Locate the "terms of use" link, which resolves to /ftp/legal.md. **3.** Navigate to the parent path: /ftp/ **4.** Observe directory listing/contents including acquisitions.md. **5.** Download and review the file to confirm its contents are non-public.

Proof	
Recommendation	**Who: GTST student (Yabsira Dawit)** • Remove non-public files from any web-accessible directory; store internal documents outside the web root. • Disable directory listing on all web-facing file paths. • Apply authentication and authorization checks to any endpoint intended for internal use only. • Conduct a full inventory of web-root content to identify any other inadvertently exposed files.

Summary of Findings

The following list summarizes all items requiring remediation, ordered by severity.

Ref	Description	CVSS Score	Severity
JS-01	SQL Injection — Authentication Bypass	9.1	**CRITICAL**
JS-02	Broken Authentication — Brute-Forceable Password	9.1	**CRITICAL**
JS-03	Insecure Direct Object Reference (IDOR)	6.5	**MEDIUM**
JS-04	Broken Access Control — Unauthorized Deletion	6.5	**MEDIUM**
JS-05	Persistent Cross-Site Scripting (XSS)	6.1	**MEDIUM**
JS-06	Sensitive Information Disclosure	5.3	**MEDIUM**

Conclusion

The penetration test conducted by Geez Security against OWASP Juice Shop identified six vulnerabilities spanning authentication, access control, and injection vulnerability classes. The two critical findings — SQL injection-based authentication bypass and brute-forceable administrator credentials — present the most significant risk, as either alone grants an unauthenticated attacker full administrative control of the application.

The remaining medium-severity findings, while less immediately catastrophic, compound the risk picture: once inside the application (whether through the critical findings above or a legitimate low-privilege account), an attacker can pivot to access other users' data, delete content without authorization, persist malicious scripts for later execution, and access non-public business files.

All identified issues are exploitable through common, well-documented attack techniques and require no custom exploit development. Geez Security strongly recommends prioritizing remediation of the CRITICAL findings immediately, with the MEDIUM findings addressed in the next regular release cycle. Geez Security is available for consultation and re-testing following remediation to confirm the identified vulnerabilities have been properly addressed.

Appendices

Appendix A: Tools and Techniques Used

- Web Application Testing: Burp Suite (Community/Professional)
- Browser-Based Analysis: Browser DevTools
- Automated Attack Tooling: Burp Suite Intruder
- Reference Wordlists: SecLists (Common-Credentials)

Appendix B: Glossary of Terms

- SQL Injection (SQLi): An attack where malicious SQL code is inserted into an input field to manipulate or access a backend database.
- Cross-Site Scripting (XSS): An attack in which malicious scripts are injected into content served by an otherwise trusted application.
- Insecure Direct Object Reference (IDOR): An access control flaw where an application exposes a reference to an internal object (e.g. a database ID) without verifying the requester is authorized to access it.
- Broken Access Control: A class of vulnerability where an application fails to properly restrict what authenticated users are allowed to do.
- CVSS (Common Vulnerability Scoring System): An open industry standard for rating the severity of security vulnerabilities on a 0–10 scale.

Appendix C: Vulnerability Severity Level Definitions

- Critical: A vulnerability that could lead to full system compromise; immediate action is required.
- High: A serious vulnerability that could allow unauthorized access or significant disruption.
- Medium: A vulnerability that presents real risk but may require certain conditions to exploit.
- Low: A minor vulnerability, unlikely to be exploited alone but worth remediating as best practice.

Appendix D: Evidence

Screenshot evidence for each finding (referenced inline above) is maintained in the assessment's supporting evidence package and available on request. Evidence includes Burp Suite request/response captures, browser screenshots of successful exploitation, and Intruder attack logs for JS-01 through JS-06.